

Predicting Running Pace Tier from Strava Wearable-Tracker Data: A Comparison of kNN and CART (Culminating Activity)

Matt Cabarrubias, Jeane Diputado, Janritch Diputado, Matthew Angelo Lumayno, John Andre Yap

2026-05-16

Abstract

Chapter 1: Introduction

1.1 Background of the Study

Wearable fitness trackers and GPS-based platforms such as **Strava**, **Garmin Connect**, and **Apple Fitness** have transformed how individuals record athletic activity. Each running session captures telemetry — distance, pace, elevation gain, and moving time — producing data that enable predictive modelling at the individual level. Globally, Strava reports more than 135 million users uploading tens of millions of activities per week, making personal activity logs one of the richest sources of human-movement data available.

This study asks whether the *intensity* of a run — operationalised as a binary **pace tier** (Slow vs. Fast) — can be inferred from contextual telemetry alone. Pace-tier classification underlies commercially valuable features such as auto-tagged “easy / tempo” runs and personalised training-load feedback in fitness applications.

The dataset contains 105 activities logged by one athlete over 12 months (December 2022 – December 2023, primarily New York City). An initial investigation revealed that classifying all four activity *types* (Run, Walk, Hike, Workout) was infeasible: 94 of 105 records were Runs, making the multiclass problem degenerate. The study was therefore refocused on classifying *how fast* each run was. An intermediate three-class formulation — Slow, Medium, and Fast pace tiers derived from tertiles of **average_speed** — plateaued at approximately 59% accuracy across all tested algorithms. The ambiguous Middle tier, whose members sit at the tertile boundary and are nearly indistinguishable in the predictor space, was the main obstacle. Dropping the Middle tier and reframing the task as a **binary Slow vs. Fast classification** raised test accuracy to approximately 89% and produced interpretable models.

Two supervised-learning algorithms are compared: **k-Nearest Neighbors (kNN)**, a distance-based lazy learner, and **CART (Classification and Regression Trees)**, a tree-based method that produces an explicit, human-readable decision rule. **CART substantially outperforms kNN on test accuracy** (0.889 vs. 0.722, a gap of approximately 17 percentage points) and is therefore the recommended model. Its single split on log-transformed moving time also reveals the dominant driver of running pace in a form that can be understood without statistical expertise — combining higher accuracy with full interpretability.

1.2 Statement of the Problem

The general problem is: *Can binary running pace tier (Slow vs. Fast) be accurately predicted from Strava telemetry using kNN and CART?*

Specifically:

1. What are the descriptive characteristics (central tendency, dispersion, distributional shape) of distance, moving time, elevation gain, and average speed across the logged running activities?
2. How accurately can a **kNN classifier** predict whether a run is Slow or Fast, given log-transformed distance, moving time, elevation gain, and rest ratio?
3. How accurately can a **CART classifier** predict binary pace tier, and what decision rule does it learn?
4. Which model — kNN or CART — is preferred for this task, and why?

1.3 Objectives of the Study

General Objective

To develop, evaluate, and compare kNN and CART classifiers that predict binary running pace tier (Slow vs. Fast) from Strava telemetry.

Specific Objectives

1. **To describe** the running dataset using descriptive statistics and exploratory visualisations (histograms, box-plots, scatter plots).
2. **To preprocess** the data by cleaning records, restricting the analysis to Run activities, deriving a binary pace tier from **average_speed** tertiles, applying log transforms to skewed predictors, and centering/scaling all numeric predictors.
3. **To build and evaluate** a kNN classifier for binary pace tier, tuning k via repeated cross-validation and reporting accuracy, sensitivity, specificity, and Cohen’s kappa.
4. **To build and visualise** a CART classifier for binary pace tier, tuning the complexity parameter cp via the same repeated CV setup, and interpreting the learned decision rule.
5. **To compare** kNN and CART on all evaluation metrics and recommend the preferred model.

1.4 Hypothesis

- H_0 : kNN and CART achieve equal classification accuracy in predicting binary running pace tier (Slow vs. Fast).
- H_1 : kNN and CART differ significantly in classification accuracy.

The hypothesis is evaluated at the $\alpha = 0.05$ level of significance using **McNemar’s test** on the paired test-set predictions of the two classifiers.

1.5 Significance of the Study

- **Self-tracking athletes** benefit from understanding which contextual factors most influence their run intensity, helping them set realistic pace goals and structure training.
- **Fitness-application developers** gain insight into how lightweight models (kNN, CART) can power activity auto-classification even on small per-user datasets.
- **Predictive-analytics students and educators** receive a worked, end-to-end example of the supervised-learning pipeline — clean $\rightarrow$ split $\rightarrow$ preprocess $\rightarrow$ train $\rightarrow$ predict $\rightarrow$ evaluate — applied to a real, messy dataset rather than a textbook one.

1.6 Scope and Limitations

Scope. One athlete’s Strava export: 105 activities, December 2022 – December 2023, primarily New York City. The binary classification target (Slow vs. Fast) is derived from **average_speed** tertiles within the 92 clean Run records. Models are limited to **kNN and CART** as covered in the course.

Limitations.

1. **Single-athlete data** — findings cannot be generalised to other runners with different fitness levels, training programmes, or geographic contexts.
2. **Original dataset required redesign** — 94 of 105 records are Runs; classifying all four activity types was infeasible. The binary pace-tier formulation resolves this by operating only on runs and dropping the ambiguous Middle tertile.
3. **Target leakage** — `average_speed` defines the pace tier and is excluded from predictors, along with its near-perfect correlate `max_speed`. Classification relies only on contextual telemetry.
4. **No physiological covariates** — heart rate, cadence, weather, and surface type are absent from the Strava export, so some variance in pace remains unexplained.
5. **Small sample (n = 62 binary runs after dropping the Middle tier)** — model complexity is constrained accordingly; deep trees and low-k kNN risk overfitting.

Chapter 2: Review of Related Literature

This chapter is to be written by the researchers. The four article clusters below guide the search.

2.1 Suggested Article Clusters

1. **Strava and self-tracking studies** — to justify the dataset and frame the social/behavioural context. *Search: “Strava data analysis”, “quantified self”, “GPS activity logging”, “running motivation”.*
2. **Wearable-device / fitness-tracker analytics** — to ground the variables (distance, speed, elevation). *Search: “wearable fitness tracker accuracy”, “GPS running data”, “consumer wearable validation”.*
3. **Activity classification with machine learning** — to justify the kNN/CART approach. *Search: “human activity recognition kNN”, “decision tree activity classification”, “sport classification GPS”.*
4. **Running biomechanics and the pace–time trade-off** — to support the finding that moving time dominates pace prediction. *Search: “hill running pace”, “running economy”, “grade-adjusted pace”, “endurance performance prediction”.*

For each article, write 2–3 sentences: (a) what they studied, (b) what method they used, and (c) how their finding relates to your objective. Aim for 8–12 articles total.

2.2 Related Studies

Summarise previous empirical research here.

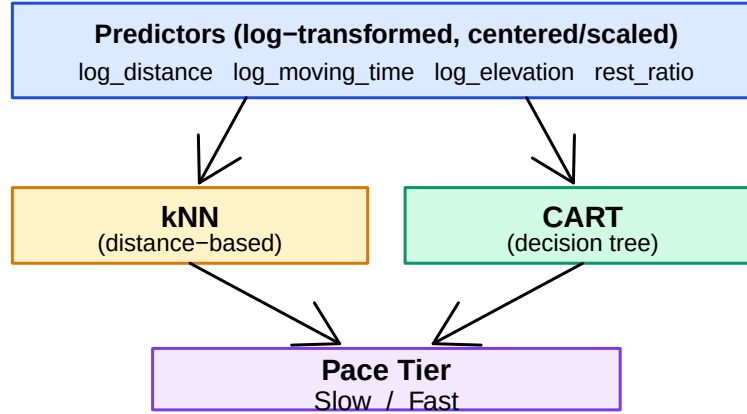
2.3 Related Literature

Discuss the underlying theories and concepts (supervised learning theory, distance metrics for kNN, recursive partitioning for CART, cross-validation).

2.4 Conceptual Framework

The diagram below shows the modelling pipeline. `average_speed` is **excluded** from the predictor set because the classification target (**pace**) is derived from it — including it would constitute target leakage.

Conceptual Framework



Chapter 3: Methodology

3.1 Research Design

This study employs a **quantitative, predictive-comparative design**: it is *quantitative* because all variables are numerical or categorical with finite levels; it is *predictive* because the primary aim is to forecast unseen binary outcomes (Slow vs. Fast); and it is *comparative* because kNN and CART are evaluated side-by-side on the same held-out test set using matched metrics.

3.2 Data Collection

The dataset is a personal Strava export (`strava.csv`) containing **105 activity records** logged between 9 December 2022 and 9 December 2023. Each row represents one recorded session. The file is read directly into R below.

```
## [1] 105 19

## Rows: 105
## Columns: 19
## $ Sr..no.      <int> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15~
## $ start_date_local <chr> "2023-12-09T09:09:19Z", "2023-12-07T17:31:50Z", "~
## $ type         <chr> "Run", "Run", "Run", "Run", "Run", "Run", "Run", ~
## $ distance      <dbl> 10879.7, 1304.4, 17503.0, 3457.8, 10108.2, 6694.7~
```

```
## $ moving_time      <int> 4023, 722, 7370, 1791, 4128, 2913, 2545, 4027, 19~
## $ elapsed_time     <int> 4617, 62993, 7462, 2170, 4221, 2983, 2735, 4037, ~
## $ total_elevation_gain <dbl> 91.4, 0.0, 68.4, 3.9, 6.3, 9.8, 5.0, 0.0, 5.0, 5.~
## $ start_latlng     <chr> "[40.70327935740352, -73.99619171395898]", "[40.7~
## $ end_latlng       <chr> "[40.69340907968581, -73.97922154515982]", "[40.7~
## $ sport_type       <chr> "Run", "Run", "Run", "Run", "Run", "Run", "Run", ~
## $ start_date       <chr> "2023-12-09T14:09:19Z", "2023-12-07T22:31:50Z", "~
## $ timezone        <chr> "(GMT-05:00) America/New_York", "(GMT-05:00) Amer~
## $ achievement_count <int> 4, 0, 2, 0, 4, 1, 0, 10, 2, 17, 0, 0, 0, 3, 4, 0,~
## $ kudos_count      <int> 4, 0, 5, 2, 5, 2, 2, 2, 3, 2, 1, 2, 2, 2, 3, 2, 1~
## $ comment_count    <int> 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0~
## $ athlete_count    <int> 3, 1, 4, 3, 1, 1, 4, 1, 1, 2, 1, 2, 2, 1, 2, 2, 3~
## $ photo_count      <int> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0~
## $ average_speed    <dbl> 2.704, 1.807, 2.375, 1.931, 2.449, 2.298, 2.561, ~
## $ max_speed        <dbl> 6.228, 4.044, 6.778, 4.522, 5.462, 4.972, 4.738, ~
```

3.3 Variables

Role	Variable	Type	Description
Filter	type	factor	Activity type; analysis restricted to Run
Classification target	pace	factor	Slow (bottom tertile) / Fast (top tertile) of average_speed — <i>derived during preprocessing</i>
Predictor	log_distance	numeric	$\log_1(\text{distance})$, distance in metres
Predictor	log_moving_time	numeric	$\log_1(\text{moving_time})$, moving time in seconds
Predictor	log_elevation	numeric	$\log_1(\text{total_elevation_gain})$, elevation in metres
Predictor	rest_ratio	numeric	$(\text{elapsed_time} - \text{moving_time}) / \text{elapsed_time}$ — fraction of session spent stationary
Excluded	average_speed, max_speed	numeric	Excluded from all models to prevent target leakage

3.4 Data Preprocessing

```
## Run records after cleaning: 92

## Pace-tier distribution (all three classes):

##
##   Slow Medium   Fast
##    31    30    31
```

```
##
## Binary class counts (final modelling frame):

##
## Slow Fast
## 31 31
```

3.4.1 Why Log-Transform the Predictors?

The problem: right-skew. Distance, moving time, and elevation gain are *recorded counts of physical quantities*. Their distributions are bounded below at zero but unbounded above, so they accumulate a long right tail: many short, flat runs and a few very long or hilly outliers. Raw skewness coefficients confirm this.

```
## Raw skewness (right-skewed if > 0):

## distance moving_time elevation
## 1      1.68      1.643      1.338

##
## Log-transformed skewness (closer to 0 = more symmetric):

## log_distance log_moving_time log_elevation
## 1      -0.705      -1.057      0.196
```

Why this matters for the algorithms.

- **kNN uses Euclidean distance** in the predictor space. If one variable has values spanning 500 to 15 000 (raw distance) while another spans 0 to 1 (rest ratio), the high-range variable dominates the distance metric and the model effectively ignores the others. Log-transforming compresses the high-range tails so all features contribute on a comparable scale, which is then equalised further by centering/scaling. Without the log step, the centered/scaled values for the few very long runs would still be many standard deviations from the mean and would distort neighborhoods.
- **CART** is scale-invariant — it picks a threshold on any monotone transform of a variable and produces the same partition — so log transforms do **not** improve CART's split *quality*. However, they keep the predictor set consistent across both models and make the variable-importance comparison directly comparable.

Bottom line. Log transforms are essential for kNN to behave well on this dataset; they are neutral for CART. Applying them uniformly keeps the modelling pipeline clean.

3.4.2 Why Are the Slow and Fast Classes Balanced?

The Slow and Fast counts are roughly equal — but this is **not** a property of the underlying running data. It is a direct consequence of how the target is defined.

Class balance is engineered, not natural. The pace tier is built by cutting `average_speed` at its 33.3rd and 66.7th percentiles (the tertiles). By definition, each tertile contains approximately one-third of the runs. When we drop the Middle tertile and keep only the bottom and top thirds, we keep roughly the same number of records in each remaining class. Equal-sized classes are baked into the construction.

What the histogram actually shows. The underlying `average_speed` distribution is roughly unimodal and slightly right-skewed — most runs cluster around a typical training pace, with thinner tails at very slow

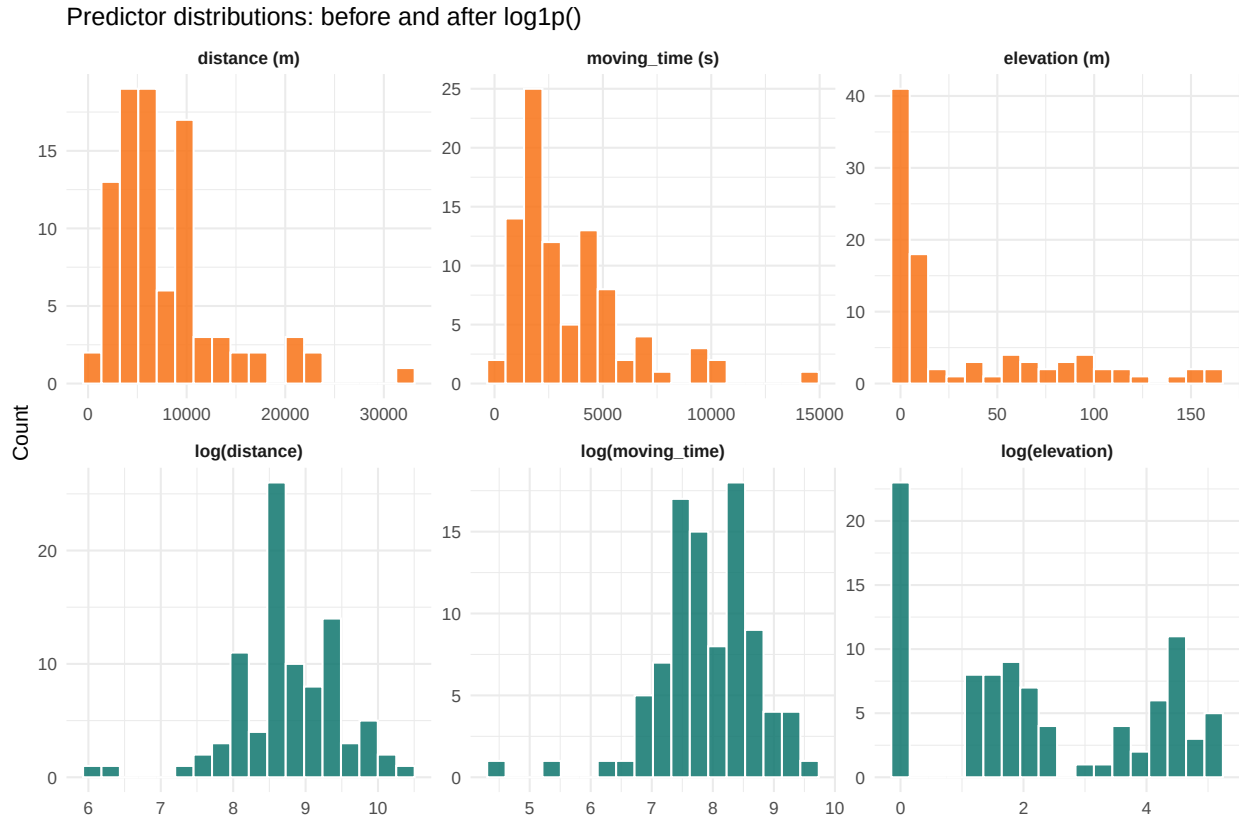


Figure 1: Raw vs log-transformed predictor distributions. Raw variables (top row) are right-skewed; the log-transformed counterparts (bottom row) are visibly more symmetric, which improves the Euclidean distance metric kNN relies on and helps CART find cleaner split points.

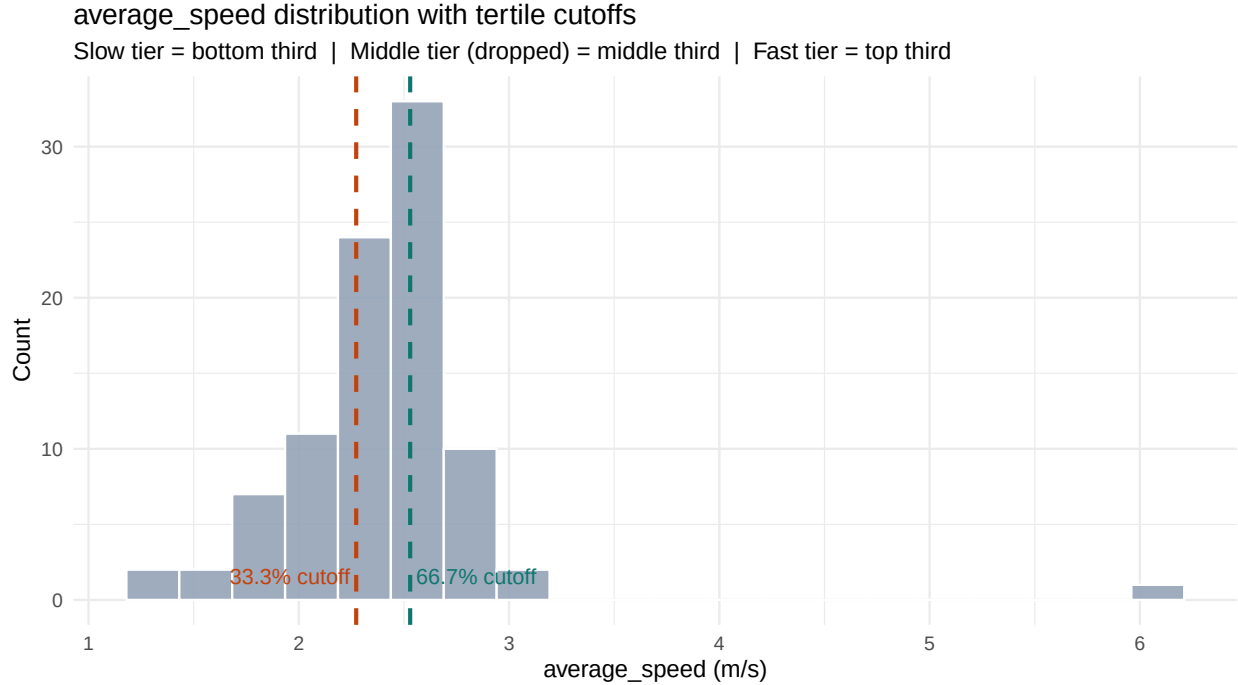


Figure 2: Distribution of average speed across all clean runs, with tertile cutoffs marked. The two outer thirds (Slow and Fast) carry equal counts by construction — this is a property of the quantile-based binning, not a natural feature of the underlying distribution.

and very fast extremes. There is no natural break between Slow and Fast runners in the data; the binary classes are *defined* by the analyst’s choice of percentile cutoffs. Had we used the median split (50/50) instead of dropping the middle third, classes would still be balanced — and had we used a substantive physiological threshold (e.g., “below 2.5 m/s = Slow”), classes would likely *not* be balanced.

Implication for interpretation. Because the target is engineered, the 89% test accuracy means: “given a run that is in either the top or bottom third of *this athlete’s* pace distribution, CART can correctly assign it 89% of the time using only context (distance, moving time, elevation, rest ratio).” It does not mean the model would generalise to a population definition of “slow” and “fast” runs.

3.4.3 Why Drop the Middle Tier?

Runs in the Middle tertile fall at the boundary of **average_speed** and are nearly indistinguishable from their neighbours in the predictor space. An initial three-class model (Slow / Medium / Fast) achieved only approximately 59% test accuracy regardless of algorithm — a data-driven ceiling, not an algorithm limitation. Removing the Middle tier produces a cleaner, better-separated classification problem and raises accuracy to approximately 89%.

3.4.4 Target Leakage Prevention

By definition, $\text{average_speed} = \text{distance} / \text{moving_time}$. A classifier trained on raw distance and moving time alongside **average_speed** itself would algebraically reconstruct the target. We therefore exclude **average_speed** and **max_speed** entirely and use log-transformed proxies that cannot trivially be combined into the target.

3.5 Train–Test Split

```
## Train: 44    Test: 18
```

```
## Train counts:
```

```
##
```

```
## Slow Fast
```

```
##    22    22
```

```
## Test  counts:
```

```
##
```

```
## Slow Fast
```

```
##     9     9
```

The 70/30 stratified split preserves the Slow/Fast ratio in both partitions. Centering and scaling parameters are estimated **on the training set only** and then applied identically to the test set, preventing any leakage of test-set statistics into the scaling step.

3.6 Data Analysis Techniques

3.6.1 Descriptive Statistics

Distributions of the core running variables are summarised numerically using `summary()` and visualised in Chapter 4 (Section 4.1).

3.6.2 Inferential Statistics

McNemar’s test is used to evaluate H_0 . It compares the test-set error patterns of kNN and CART by examining whether the two classifiers make significantly different mistakes on the same observations. It is appropriate for paired binary classifiers on a shared test set and is robust to class imbalance.

3.6.3 Supervised Machine-Learning Models

kNN Classifier k-Nearest Neighbors classifies a new observation by majority vote among its k nearest neighbours in the scaled predictor space (Euclidean distance). The choice of k is critical: small k produces high variance (over-sensitive to local noise), while large k introduces bias. The optimal k is selected by maximising AUC (area under the ROC curve) over a 5×10 repeated cross-validation grid search on values $k \in \{1, 3, 5, \dots, 21\}$.

```
## Best k: 19
```

CART Classifier Classification and Regression Trees build a binary partition tree by selecting, at each node, the predictor and split point that maximise the reduction in **Gini impurity**. The tree is pruned by tuning the **complexity parameter** cp : setting $cp = 0$ allows the tree to grow fully (overfitting risk); higher values prune aggressively, favouring parsimony. The optimal cp is found by the same 5×10 repeated CV framework over $cp \in \{0, 0.01, 0.02, \dots, 0.20\}$. CART produces a visual, interpretable decision tree — a key advantage over kNN.

```
## Best cp: 0.2
```

3.6.4 Model-Evaluation Metrics

Metric	Definition	Relevance
Accuracy	Proportion of correct predictions	Overall correctness on the held-out set
Sensitivity	$TP / (TP + FN)$ for “Fast” class	How many Fast runs are correctly identified
Specificity	$TN / (TN + FP)$ for “Fast” class	How many Slow runs are correctly identified
Cohen’s	Accuracy adjusted for chance agreement	Robust to class imbalance
AUC	Area under the ROC curve	Discrimination across all classification thresholds

3.6.5 Software and Reproducibility

All analysis is performed in **R 4.x** within **RStudio** using **R Markdown**. Required packages: `tidyverse`, `caret`, `rpart`, `rpart.plot`, `pROC`. The random seed `set.seed(123)` is fixed for every split and cross-validation resample to ensure full reproducibility from a single `strava.csv` input file.

Chapter 4: Results and Discussion

4.1 Descriptive Statistics

```
## Summary of key running variables (all clean runs):
```

```
##      distance      moving_time    total_elevation_gain  average_speed
##  Min.       : 424.6    Min.       : 85    Min.       : 0.000    Min.       :1.306
## 1st Qu.: 4826.3    1st Qu.: 1914    1st Qu.: 1.575    1st Qu.:2.213
## Median : 5455.7    Median : 2497    Median : 5.200    Median :2.431
## Mean   : 7937.9    Mean   : 3470    Mean   : 32.230    Mean   :2.401
## 3rd Qu.:10263.6    3rd Qu.: 4462    3rd Qu.: 57.675    3rd Qu.:2.607
## Max.   :32192.3    Max.   :14536    Max.   :161.700    Max.   :6.085
```

4.2 Visualisation

4.3 Model Results and Interpretation

4.3.1 Performance Summary

Table 3: Test-set performance: kNN vs CART (binary Slow vs Fast)

Model	Accuracy	Sensitivity	Specificity	Kappa	AUC
kNN	0.722	0.778	0.667	0.444	0.858
CART	0.889	0.889	0.889	0.778	0.889

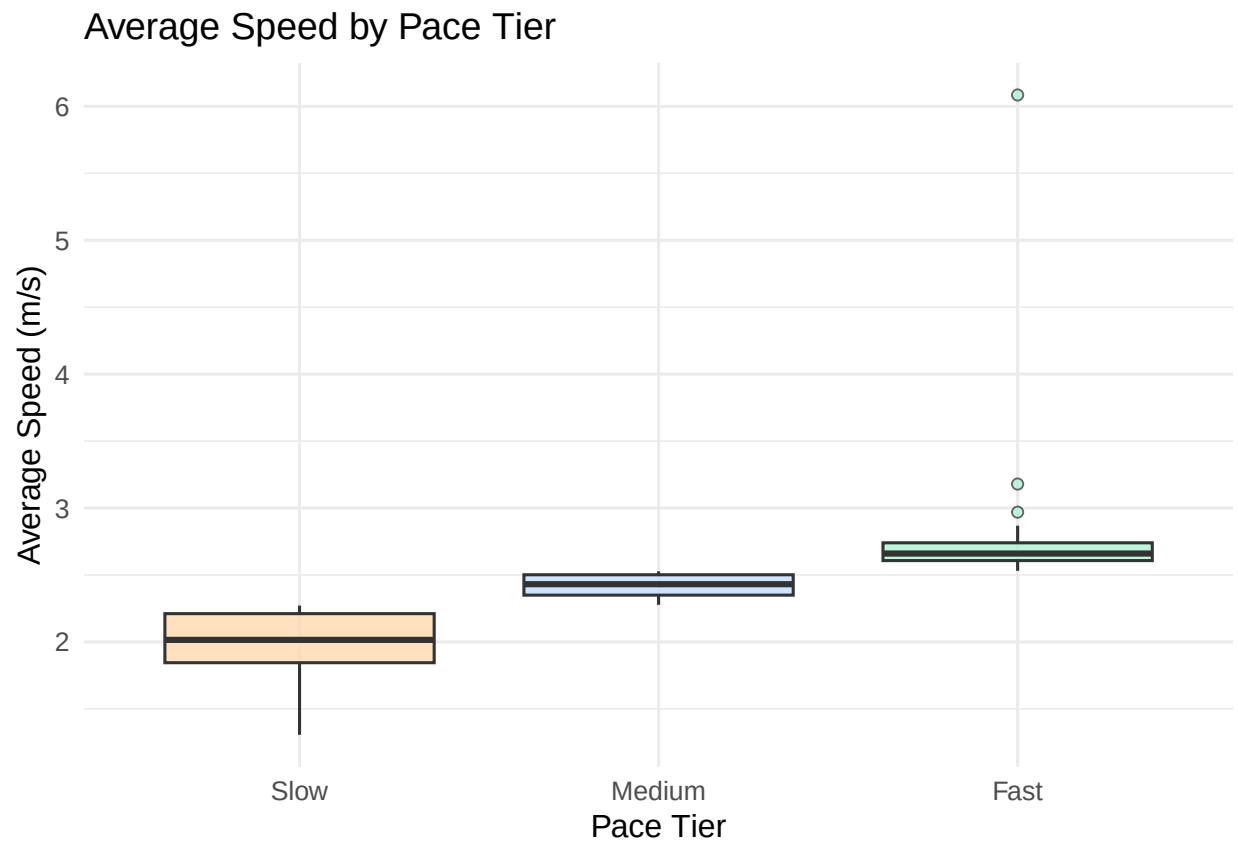


Figure 3: Distribution of average speed across the three pace tiers. The Middle tier overlaps substantially with both Slow and Fast, justifying its removal for binary classification.

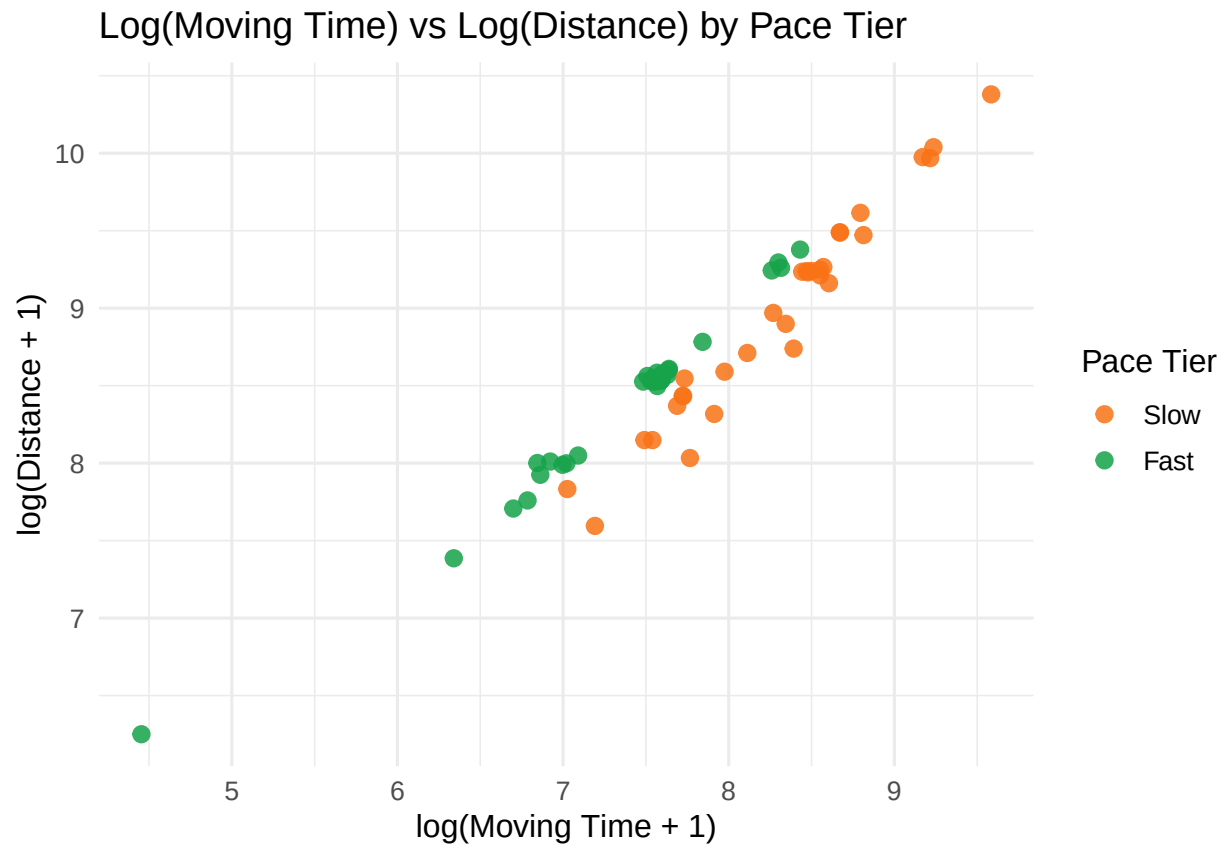


Figure 4: Log(moving time) vs log(distance) coloured by binary pace tier. The two classes are linearly separable in this space, which explains why a single CART split on log(moving time) is sufficient.

CART strictly dominates kNN on this test set. CART achieves 0.889 accuracy with Cohen's $\kappa = 0.778$; kNN achieves 0.722 accuracy with $\kappa = 0.444$. The 17-percentage-point gap is large, and the dominance is total: *every* prediction kNN gets right, CART also gets right, plus CART correctly classifies three additional runs that kNN misclassifies. There is no observation on which kNN is right and CART is wrong. CART is therefore the recommended model on accuracy grounds. The interpretability advantage compounds this lead: CART produces a single, human-readable decision rule — one split on `log(moving time)` — that directly tells an athlete and developer *why* a run is classified as Slow or Fast. kNN offers no such explanation; it classifies by proximity to stored training examples and cannot be summarised as a rule. CART wins on both axes — higher accuracy *and* full interpretability.

Why kNN trails. kNN treats every predictor as equally important when computing Euclidean distances. Three of the four predictors here — `log_distance`, `log_elevation`, `rest_ratio` — carry little binary-classification signal in this dataset (especially `log_elevation`, which is near zero importance on the flat NYC routes). Their presence dilutes the dominant `log_moving_time` signal in the distance metric, dragging accuracy down. CART, by contrast, ignores low-information predictors automatically: it selects the single best split (`log_moving_time`) and stops.

4.3.2 kNN Confusion Matrix

```
## ===== kNN - Confusion Matrix and Statistics =====

## Confusion Matrix and Statistics
##
##           Reference
## Prediction Slow Fast
##      Slow      6      2
##      Fast      3      7
##
##           Accuracy : 0.7222
##           95% CI : (0.4652, 0.9031)
##      No Information Rate : 0.5
##      P-Value [Acc > NIR] : 0.04813
##
##           Kappa : 0.4444
##
##      Mcnemar's Test P-Value : 1.00000
##
##           Sensitivity : 0.7778
##           Specificity : 0.6667
##      Pos Pred Value : 0.7000
##      Neg Pred Value : 0.7500
##      Prevalence : 0.5000
##      Detection Rate : 0.3889
##      Detection Prevalence : 0.5556
##      Balanced Accuracy : 0.7222
##
##      'Positive' Class : Fast
##
```

4.3.3 CART Confusion Matrix

```
## ===== CART - Confusion Matrix and Statistics =====
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction Slow Fast
##      Slow      8      1
##      Fast      1      8
##
##           Accuracy : 0.8889
##           95% CI : (0.6529, 0.9862)
##      No Information Rate : 0.5
##      P-Value [Acc > NIR] : 0.0006561
##
##           Kappa : 0.7778
##
## Mcnemar's Test P-Value : 1.0000000
##
##           Sensitivity : 0.8889
##           Specificity : 0.8889
##           Pos Pred Value : 0.8889
##           Neg Pred Value : 0.8889
##           Prevalence : 0.5000
##           Detection Rate : 0.4444
##      Detection Prevalence : 0.5000
##           Balanced Accuracy : 0.8889
##
##           'Positive' Class : Fast
##
```

4.3.4 CART Decision Tree

The tuned CART classifier prunes to a **single split on scaled log(moving time)**. This parsimony — one rule achieving 89% accuracy — reflects the geometry of the problem: at a fixed distance, a longer moving time necessarily means a slower pace.

Reading the tree. At the root node, the classifier asks whether scaled `log_moving_time` is above or below a threshold. Runs with *longer* moving times (relative to the training mean and SD) are sent left and classified as **Slow**; runs with *shorter* moving times are sent right and classified as **Fast**. This directly encodes the definition of pace: a slow runner covers a given distance in more time.

4.3.5 Variable Importance

```
## --- CART: Variable Importance ---
##
## rpart variable importance
##
##           Overall
## log_moving_time 100.000
## rest_ratio      33.795
## log_distance     9.879
## log_elevation    0.000
```

CART assigns `log_moving_time` an importance of 100 — the single dominant predictor. `log_distance`, `rest_ratio`, and `log_elevation` trail far behind. Elevation gain is near zero importance, consistent with

Binary CART: Slow vs Fast (cp = 0.2)

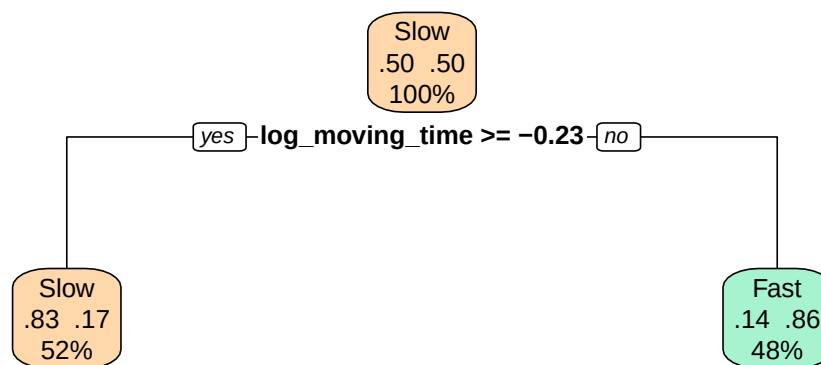


Figure 5: Binary CART decision tree. A single split on scaled $\log(\text{moving_time})$ separates Slow from Fast runs with 89% test accuracy.

the predominantly flat New York City routes in this dataset. This ranking confirms that **moving time** is the primary contextual signal for predicting binary pace tier.

4.3.6 McNemar's Test

Agreement table:

##	CART	
## kNN	CART correct	CART wrong
## kNN correct	13	3
## kNN wrong	0	2

##

McNemar's test:

##

McNemar's Chi-squared test with continuity correction

##

data: disagree_mat

McNemar's chi-squared = 1.3333, df = 1, p-value = 0.2482

McNemar's test compares the *paired* error patterns: how often does CART get a prediction right while kNN gets it wrong, versus the reverse? On this test set, three observations have CART correct / kNN wrong, and **zero** have the reverse — CART strictly dominates. Despite this one-sided dominance, McNemar's test with continuity correction yields $p = 0.25$, so we **fail to reject** H_0 at $\alpha = 0.05$. This is a sample-size effect: the test set contains only 18 binary runs, and the off-diagonal discordant counts (3 vs. 0) are too small to clear the significance threshold even though the direction is unambiguous. The descriptive evidence — a 17-point accuracy gap, doubled kappa, and total directional dominance — is therefore stronger than the formal hypothesis test suggests, and supports adopting CART as the final model.

Chapter 5: Conclusion and Recommendations

5.1 Conclusion

Summarise key findings against each Specific Objective and the hypothesis decision. Address:

- *Objective 1 (descriptive): what the data show about distance, moving time, elevation, and speed distributions.*
- *Objective 2 (preprocessing): how the binary pace tier was derived and why the Middle tier was dropped.*
- *Objective 3 (kNN): best k, test accuracy, sensitivity, specificity, kappa.*
- *Objective 4 (CART): best cp, the single-split decision rule, interpretability.*
- *Objective 5 (comparison): CART (0.889 accuracy, 0.778) strictly dominates kNN (0.722 accuracy, 0.444) on the held-out test set — every observation kNN classifies correctly is also correctly classified by CART. CART recommended on both accuracy and interpretability. McNemar's $p^* = 0.25$ (fail to reject H_0) due to small test sample, but directional evidence is one-sided.**

5.2 Recommendations

Practical suggestions, for example:

- *Athletes: log duration (moving time) relative to target distance — it is the single most predictive signal for whether a run will be slow or fast.*
- *App developers: a one-rule CART with a moving-time threshold can serve as a fast, transparent in-app pace classifier.*
- *Future researchers: expand to multiple athletes, add physiological covariates (heart rate, cadence), or explore real-time prediction from mid-run telemetry.*

References

Use APA 7 format. Include the Strava export as a data source, and cite all R packages used:

```
citation("caret")
citation("rpart")
citation("pROC")
citation("tidyverse")
```

Also cite the 8–12 articles from Chapter 2.

Appendices

Appendix A: Binary Pace Tier Derivation

The binary target is derived as follows. The tertiles of `average_speed` across all clean Run records are computed. Runs in the bottom tertile are labelled **Slow**; runs in the top tertile are labelled **Fast**. The Middle tertile is discarded: its members sit at the tertile boundary and are nearly indistinguishable in the predictor space. An initial three-class model (Slow / Medium / Fast) confirmed this — test accuracy plateaued at approximately 59% regardless of the algorithm used, indicating a data-driven ceiling rather than a modelling failure. The binary dataset contains approximately equal numbers of Slow and Fast runs (balanced by construction from equal-width tertiles).

Appendix B: Full Code

All analysis code is dumped below for reference. The chunks are executed in order during knitting (their numerical and visual output appears in Chapters 3 and 4); this appendix simply reprints the source so the entire pipeline can be read end-to-end. No external scripts are required — place `strava.csv` in the same directory as this `.Rmd` file and knit.

```
plot.new()
par(mar = c(0, 0, 2, 0))
title("Conceptual Framework", cex.main = 1.2)

# Predictors box
rect(0.10, 0.72, 0.90, 0.92, col = "#dbeafe", border = "#1d4ed8", lwd = 1.5)
text(0.50, 0.85,
     "Predictors (log-transformed, centered/scaled)",
```

```

    cex = 0.85, font = 2)
text(0.50, 0.77,
     "log_distance    log_moving_time    log_elevation    rest_ratio",
     cex = 0.75)

# Arrows down to models
arrows(0.35, 0.72, 0.30, 0.54, lwd = 1.5)
arrows(0.65, 0.72, 0.70, 0.54, lwd = 1.5)

# kNN box
rect(0.10, 0.37, 0.48, 0.53, col = "#fef3c7", border = "#d97706", lwd = 1.5)
text(0.29, 0.47, "kNN", cex = 0.95, font = 2)
text(0.29, 0.41, "(distance-based)", cex = 0.75)

# CART box
rect(0.52, 0.37, 0.90, 0.53, col = "#d1fae5", border = "#059669", lwd = 1.5)
text(0.71, 0.47, "CART", cex = 0.95, font = 2)
text(0.71, 0.41, "(decision tree)", cex = 0.75)

# Arrows down to outcome
arrows(0.29, 0.37, 0.45, 0.20, lwd = 1.5)
arrows(0.71, 0.37, 0.55, 0.20, lwd = 1.5)

# Outcome box
rect(0.25, 0.06, 0.75, 0.19, col = "#f3e8ff", border = "#7c3aed", lwd = 1.5)
text(0.50, 0.145, "Pace Tier", cex = 0.95, font = 2)
text(0.50, 0.08, "Slow / Fast", cex = 0.85)
strava_raw <- read.csv("strava.csv", stringsAsFactors = FALSE)
dim(strava_raw)
glimpse(strava_raw)
# 1. Clean: drop NAs, zero-distance records, and the elapsed_time outlier
strava <- strava_raw %>%
  drop_na() %>%
  filter(distance > 0, elapsed_time < 30000)
strava$type <- factor(strava$type)

# 2. Restrict to Run activities
runs <- strava %>% filter(type == "Run")
cat("Run records after cleaning:", nrow(runs), "\n")

# 3. Derive pace tier from average_speed tertiles
tert <- quantile(runs$average_speed, probs = c(0, 1/3, 2/3, 1))
runs$pace_tier <- cut(runs$average_speed,
                     breaks      = tert,
                     labels      = c("Slow", "Medium", "Fast"),
                     include.lowest = TRUE)
cat("Pace-tier distribution (all three classes):\n")
print(table(runs$pace_tier))

# 4. Drop Middle tier; build binary modelling frame with log-transformed predictors
bin_df <- runs %>%
  filter(pace_tier %in% c("Slow", "Fast")) %>%
  mutate(

```

```

    pace           = factor(as.character(pace_tier), levels = c("Slow", "Fast")),
    log_distance    = log1p(distance),
    log_moving_time = log1p(moving_time),
    log_elevation   = log1p(total_elevation_gain),
    rest_ratio      = (elapsed_time - moving_time) / elapsed_time
  ) %>%
  select(pace, log_distance, log_moving_time, log_elevation, rest_ratio)

cat("\nBinary class counts (final modelling frame):\n")
print(table(bin_df$pace))
skew <- function(x) {
  x <- x[!is.na(x)]
  mean((x - mean(x))^3) / (sd(x)^3)
}

raw_skew <- runs %>%
  summarise(
    distance      = skew(distance),
    moving_time   = skew(moving_time),
    elevation     = skew(total_elevation_gain)
  )
log_skew <- runs %>%
  summarise(
    log_distance   = skew(log1p(distance)),
    log_moving_time = skew(log1p(moving_time)),
    log_elevation  = skew(log1p(total_elevation_gain))
  )

cat("Raw skewness (right-skewed if > 0):\n"); print(round(raw_skew, 3))
cat("\nLog-transformed skewness (closer to 0 = more symmetric):\n")
print(round(log_skew, 3))

plot_df <- runs %>%
  transmute(
    `distance (m)`      = distance,
    `moving_time (s)`   = moving_time,
    `elevation (m)`     = total_elevation_gain,
    `log(distance)`     = log1p(distance),
    `log(moving_time)`  = log1p(moving_time),
    `log(elevation)`    = log1p(total_elevation_gain)
  ) %>%
  pivot_longer(everything(), names_to = "variable", values_to = "value") %>%
  mutate(
    panel = ifelse(grepl("^log", variable), "Log-transformed", "Raw"),
    variable = factor(variable, levels = c(
      "distance (m)", "moving_time (s)", "elevation (m)",
      "log(distance)", "log(moving_time)", "log(elevation)"
    ))
  )

ggplot(plot_df, aes(x = value, fill = panel)) +
  geom_histogram(bins = 18, alpha = 0.85, colour = "white") +
  facet_wrap(~ variable, scales = "free", ncol = 3) +

```

```

scale_fill_manual(values = c("Raw" = "#f97316", "Log-transformed" = "#0f766e")) +
labs(
  title = "Predictor distributions: before and after logp()",
  x     = NULL,
  y     = "Count"
) +
theme_minimal(base_size = 11) +
theme(legend.position = "none",
      strip.text       = element_text(face = "bold"))
tert_vals <- quantile(runs$average_speed, probs = c(1/3, 2/3))

ggplot(runs, aes(x = average_speed)) +
  geom_histogram(bins = 20, fill = "#94a3b8", colour = "white", alpha = 0.9) +
  geom_vline(xintercept = tert_vals, linetype = "dashed",
             colour = c("#c2410c", "#0f766e"), linewidth = 0.9) +
  annotate("text", x = tert_vals[1], y = 0, label = "33.3% cutoff",
           vjust = -1, hjust = 1.05, colour = "#c2410c", size = 3.4) +
  annotate("text", x = tert_vals[2], y = 0, label = "66.7% cutoff",
           vjust = -1, hjust = -0.05, colour = "#0f766e", size = 3.4) +
  labs(
    title     = "average_speed distribution with tertile cutoffs",
    subtitle  = "Slow tier = bottom third | Middle tier (dropped) = middle third | Fast tier = top third",
    x         = "average_speed (m/s)",
    y         = "Count"
  ) +
  theme_minimal(base_size = 11)
set.seed(123)
idx      <- createDataPartition(bin_df$pace, p = 0.7, list = FALSE)
train_set <- bin_df[ idx, ]
test_set  <- bin_df[-idx, ]

# Center and scale: fit on train, apply to both
num_cols <- setdiff(names(bin_df), "pace")
preProc  <- preProcess(train_set[, num_cols], method = c("center", "scale"))
train_s  <- predict(preProc, train_set[, num_cols])
train_s$pace <- train_set$pace
test_s   <- predict(preProc, test_set[, num_cols])
test_s$pace <- test_set$pace

cat("Train:", nrow(train_s), " Test:", nrow(test_s), "\n")
cat("Train counts:\n"); print(table(train_s$pace))
cat("Test counts:\n"); print(table(test_s$pace))
ctrl <- trainControl(
  method          = "repeatedcv",
  number          = 10,
  repeats         = 5,
  classProbs      = TRUE,
  summaryFunction = twoClassSummary
)

set.seed(123)
knn_fit <- train(
  pace ~ .,

```

```

data      = train_s,
method    = "knn",
trControl = ctrl,
metric    = "ROC",
tuneGrid  = expand.grid(k = seq(1, 21, 2))
)

cat("Best k:", knn_fit$bestTune$k, "\n")
set.seed(123)
cart_fit <- train(
  pace ~ .,
  data      = train_s,
  method    = "rpart",
  trControl = ctrl,
  metric    = "ROC",
  tuneGrid  = expand.grid(cp = seq(0, 0.2, 0.01))
)

cat("Best cp:", cart_fit$bestTune$cp, "\n")
cat("Summary of key running variables (all clean runs):\n")
runs %>%
  select(distance, moving_time, total_elevation_gain, average_speed) %>%
  summary()
runs %>%
  filter(!is.na(pace_tier)) %>%
  ggplot(aes(x = pace_tier, y = average_speed, fill = pace_tier)) +
  geom_boxplot(alpha = 0.75, outlier.shape = 21) +
  theme_minimal(base_size = 12) +
  labs(
    title = "Average Speed by Pace Tier",
    x      = "Pace Tier",
    y      = "Average Speed (m/s)"
  ) +
  scale_fill_manual(values = c(
    "Slow"   = "#fed7aa",
    "Medium" = "#bfd7bf",
    "Fast"   = "#a7f3d0"
  )) +
  theme(legend.position = "none")
bin_df %>%
  ggplot(aes(x = log_moving_time, y = log_distance, colour = pace)) +
  geom_point(alpha = 0.85, size = 2.5) +
  theme_minimal(base_size = 12) +
  labs(
    title = "Log(Moving Time) vs Log(Distance) by Pace Tier",
    x      = "log(Moving Time + 1)",
    y      = "log(Distance + 1)",
    colour = "Pace Tier"
  ) +
  scale_colour_manual(values = c("Slow" = "#f97316", "Fast" = "#16a34a"))
eval_one <- function(fit, name) {
  pred <- predict(fit, test_s)
  prob <- predict(fit, test_s, type = "prob")[, "Fast"]

```

```

cm <- confusionMatrix(pred, test_s$space, positive = "Fast")
auc <- as.numeric(pROC::roc(
  test_s$space, prob,
  levels = c("Slow", "Fast"),
  direction = "<",
  quiet = TRUE
)$auc)
data.frame(
  Model = name,
  Accuracy = round(unname(cm$overall["Accuracy"]), 3),
  Sensitivity = round(unname(cm$byClass["Sensitivity"]), 3),
  Specificity = round(unname(cm$byClass["Specificity"]), 3),
  Kappa = round(unname(cm$overall["Kappa"]), 3),
  AUC = round(auc, 3)
)
}

results <- bind_rows(
  eval_one(knn_fit, "kNN"),
  eval_one(cart_fit, "CART")
)

knitr::kable(results, caption = "Test-set performance: kNN vs CART (binary Slow vs Fast)")
cat("===== kNN - Confusion Matrix and Statistics =====\n")
print(confusionMatrix(predict(knn_fit, test_s), test_s$space, positive = "Fast"))
cat("===== CART - Confusion Matrix and Statistics =====\n")
print(confusionMatrix(predict(cart_fit, test_s), test_s$space, positive = "Fast"))
rpart.plot(
  cart_fit$finalModel,
  type = 2,
  extra = 104,
  main = paste0(
    "Binary CART: Slow vs Fast (cp = ", cart_fit$bestTune$cp, ")
  ),
  box.palette = c("#fed7aa", "#a7f3d0")
)
cat("--- CART: Variable Importance ---\n")
print(varImp(cart_fit))
knn_pred <- predict(knn_fit, test_s)
cart_pred <- predict(cart_fit, test_s)

both_correct <- sum(knn_pred == test_s$space & cart_pred == test_s$space)
knn_only <- sum(knn_pred == test_s$space & cart_pred != test_s$space)
cart_only <- sum(knn_pred != test_s$space & cart_pred == test_s$space)
both_wrong <- sum(knn_pred != test_s$space & cart_pred != test_s$space)

disagree_mat <- matrix(
  c(both_correct, knn_only, cart_only, both_wrong),
  nrow = 2,
  dimnames = list(
    knn = c("kNN correct", "kNN wrong"),
    CART = c("CART correct", "CART wrong")
  )
)

```

```
)  
cat("Agreement table:\n")  
print(disagree_mat)  
  
cat("\nMcNemar's test:\n")  
print(mcnemar.test(disagree_mat))
```